

Checklist des Mesures de Sécurité

Rôle Développeur

Application de Gestion de Comptes Utilisateurs

Projet	Programmation Sécurisée
Formation	Bachelor 3 Cybersécurité
École	ESAIP La Salle
Date	Janvier 2026
Développeur	Oumar Touré
Analyste Sécurité	Joel cedric

Introduction

Ce document présente les mesures de sécurité que j'ai implémentées dans l'application. J'ai essayé de couvrir les principales vulnérabilités qu'on a vues en cours (OWASP Top 10) tout en gardant le code lisible et maintenable.

Pour chaque mesure, j'indique le fichier concerné et j'explique pourquoi j'ai fait ce choix. Y'a aussi quelques extraits de code pour montrer concrètement comment c'est fait.

1. Authentification

1.1 Hashage des mots de passe

Fichier : `src/models/User.js`

Les mots de passe sont hashés avec `bcrypt` avant d'être stockés en base. J'utilise 12 rounds de salage, c'est ce qui est recommandé actuellement (assez lent pour empêcher le brute force mais pas trop pour l'UX).

Le hashage se fait automatiquement avant chaque `save()` grâce au middleware pre-save de Mongoose. Comme ça on est sûr de jamais oublier de hasher.

```
userSchema.pre('save', async function(next) {
  if (!this.isModified('password')) return next();
  const salt = await bcrypt.genSalt(12);
  this.password = await bcrypt.hash(this.password, salt);
  next();
});
```

1.2 Tokens JWT

Fichiers : `src/utils/jwt.js`, `src/controllers/authController.js`

J'utilise JWT pour l'authentification stateless. Le token contient juste l'ID utilisateur (pas d'infos sensibles) et expire après 24h.

La clé secrète est dans une variable d'environnement (`.env`), jamais en dur dans le code. En dev y'a une clé par défaut mais en prod faut absolument la changer.

1.3 Stockage sécurisé du token

Fichier : `src/controllers/authController.js`

Au début je stockais le JWT dans `localStorage` (c'est ce qu'on voit souvent dans les tutos) mais c'est vulnérable au XSS. Maintenant le token est dans un cookie avec ces options :

Option	Valeur	Pourquoi
<code>httpOnly</code>	<code>true</code>	JavaScript peut pas lire le cookie → protège du XSS
<code>secure</code>	<code>true</code> (en prod)	Cookie envoyé que en HTTPS
<code>sameSite</code>	<code>strict</code>	Cookie pas envoyé depuis d'autres sites → protège du CSRF
<code>maxAge</code>	24h	Le cookie expire en même temps que le JWT

2. Protection contre les attaques

2.1 Injection NoSQL

Fichiers : `src/models/User.js`, `src/routes/auth.js`

Comme j'utilise MongoDB, y'a pas de SQL injection classique mais y'a quand même des risques d'injection NoSQL. Pour m'en protéger :

- J'utilise Mongoose avec des schémas typés (le champ email doit être un String, pas un objet)
- Je valide les entrées avec express-validator avant de les utiliser dans les requêtes
- Je normalise l'email en minuscules pour éviter les contournements

2.2 XSS (Cross-Site Scripting)

Fichiers : `public/js/app.js`, `src/index.js`

Deux protections complémentaires :

Côté client : J'échappe toutes les données affichées avec une fonction `escapeHtml()`. Par exemple quand j'affiche la liste des utilisateurs dans le tableau admin.

```
function escapeHtml(text) {
  if (!text) return '';
  var div = document.createElement('div');
  div.textContent = text; // textContent échappe automatiquement
  return div.innerHTML;
}
```

Côté serveur : Helmet ajoute une Content-Security-Policy restrictive qui bloque l'exécution de scripts inline ou venant de domaines non autorisés.

2.3 CSRF (Cross-Site Request Forgery)

Fichier : `src/controllers/authController.js`

La protection CSRF est assurée par le cookie `SameSite=strict`. Ça veut dire que le navigateur enverra pas le cookie si la requête vient d'un autre site.

J'aurais pu aussi mettre un token CSRF dans les formulaires mais avec `SameSite=strict` + cookie `httpOnly` c'est déjà bien protégé. Et ça simplifie le code frontend.

2.4 Brute Force

Fichier : `src/middleware/rateLimit.js`

J'ai mis un rate limiter sur la route de login : max 5 tentatives par IP toutes les 15 minutes. Après ça l'utilisateur reçoit une erreur 429.

```
const loginLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 5, // 5 essais max
  message: { error: 'Trop de tentatives. Réessayez dans 15 min.' }
});
```

Y'a aussi un rate limiter général sur l'API (100 req/min) pour éviter les abus.

3. Headers HTTP de sécurité

Fichier : `src/index.js`

J'utilise Helmet pour ajouter automatiquement les headers de sécurité recommandés. Voici ce que ça configure :

Header	Valeur	Protection
Content-Security-Policy	default-src 'self'	Bloque les scripts/styles externes
X-Frame-Options	DENY	Empêche d'embarquer le site dans une iframe (click jacking)
X-Content-Type-Options	nosniff	Empêche le navigateur de deviner le type MIME
X-XSS-Protection	1; mode=block	Protection XSS des vieux navigateurs
Strict-Transport-Security	max-age=31536000	Force HTTPS pendant 1 an

Note : J'ai dû autoriser 'unsafe-inline' pour les styles parce que sinon ça cassait certains styles CSS inline. C'est pas idéal mais c'était compliqué de tout refaire autrement.

4. Validation des entrées

4.1 Côté serveur (express-validator)

Fichier : `src/routes/auth.js`

Toutes les entrées utilisateur sont validées côté serveur AVANT d'être traitées. C'est important parce qu'on peut jamais faire confiance au client (un attaquant peut envoyer des requêtes directement sans passer par le formulaire).

Champ	Validation
Email	Format email valide + normalisation en minuscules
Mot de passe	Minimum 8 caractères
Nom	Entre 2 et 50 caractères, échappé
Rôle	Doit être 'user' ou 'admin' (whitelist)

4.2 Côté client

Fichier : `public/js/app.js`

Y'a aussi une validation côté client pour l'UX (afficher les erreurs tout de suite sans attendre la réponse serveur) mais c'est juste un bonus, la vraie validation c'est côté serveur.

5. Gestion des rôles et autorisations

Fichier : `src/controllers/authController.js`

Y'a deux rôles : 'user' (par défaut) et 'admin'. La vérification se fait dans chaque fonction admin, pas juste au niveau des routes.

J'ai aussi ajouté des protections contre les erreurs de manipulation :

- Un admin peut pas se retirer ses propres droits (sinon on pourrait se retrouver sans admin)
- Un admin peut pas se supprimer lui-même

6. Autres bonnes pratiques

Variables sensibles : Tout ce qui est secret (clé JWT, URI MongoDB) est dans le fichier .env qui est dans le .gitignore. Jamais commité.

Pas de fuite d'infos : Le mot de passe hashé est jamais renvoyé dans les réponses API. J'ai créé une méthode toSafeObject() sur le modèle User pour ça.

Messages d'erreur génériques : Quand le login échoue, je renvoie toujours "Identifiants incorrects" que ce soit l'email ou le mot de passe qui soit faux. Comme ça un attaquant peut pas savoir si un email existe dans la base.

Limite taille requêtes : express.json() est configuré avec limit: '10kb' pour éviter les attaques par déni de service avec des requêtes énormes.

7. Ce qu'on pourrait améliorer

Si j'avais eu plus de temps, j'aurais aimé ajouter :

- **2FA** (double authentification) avec TOTP ou SMS
- **Règles de complexité** pour les mots de passe (majuscules, chiffres, symboles)
- **Récupération de mot de passe** par email avec un lien temporaire
- **Logs d'audit** pour tracer toutes les actions sensibles
- **Captcha** sur les formulaires publics
- **Refresh tokens** pour renouveler la session sans re-login

Récapitulatif

Mesure	Implémenté ?	Fichier principal
Hashage mots de passe (bcrypt)	✓	src/models/User.js
JWT dans cookie httpOnly	✓	src/controllers/authController.js
Cookie SameSite strict	✓	src/controllers/authController.js
Rate limiting login	✓	src/middleware/rateLimit.js
Validation serveur	✓	src/routes/auth.js
Headers sécurité (Helmet)	✓	src/index.js
Échappement XSS	✓	public/js/app.js
Gestion des rôles	✓	src/controllers/authController.js
Messages erreur génériques	✓	src/controllers/authController.js

2FA	X	-
Complexité mot de passe	X	-